produces:

```
Hello
Hello
```

The block (the code between the braces) is part of the call to the two_times method. Within this method, yield is called two times. Each time, it invokes the code in the block, and a cheery greeting is printed.

What makes blocks interesting, however, is that you can pass parameters to them and receive values from them. For example, we could write a simple function that calculates members of the Fibonacci series up to a certain value. (The basic Fibonacci series is a sequence of integers, starting with two 1s, in which each subsequent term is the sum of the two preceding terms. The series is sometimes used in sorting algorithms and in analyzing natural phenomena.)

Continuing the example, let's say we want the method to be able to do something arbitrary with each new Fibonacci number. We can allow that by passing a block to the method, and then yielding to the block each time we identify a new Fibonacci number.

```
def fib_up_to(max)
  # parallel assignment (i1 = 1 and i2 = 1)
  i1, i2 = 1, 1
  while i1 <= max
    yield i1
    i1, i2 = i2, i1 + i2
  end
end

fib_up_to(1000) { |f| print f, " " }
puts
```

produces:

```
1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
```

In this example, the yield statement has an argument. This value is passed to the associated block, which tells the method what to do with each successive element.

In the definition of the block, the parameter list appears between vertical bars. In this instance, the variable f receives the value passed to yield and then by yield to the block, so the block prints successive members of the series. (This example also shows parallel assignment in action. We'll come back to this later on page 148.) Using the shortcut syntax we saw earlier, this block could also have been written as { print _1, " " }.

Although it is common to pass just one value to a block, this is not a requirement; a block may have any number of arguments. Blocks can use any of the argument patterns that methods use, including keyword arguments, * and ** splats, and & arguments. These patterns are discussed further in Chapter 5, More About Methods, on page 85.

Ruby provides many iterators that are available to all Ruby collections. Let's look at three: each, find, and map.

The each method is probably the simplest iterator—all it does is yield successive elements of its collection:

```
[1, 3, 5, 7, 9].each { |i| puts i }
```